

JWT Authentication Filter & OncePerRequestFilter - Study Guide

Why do we need JwtAuthenticationFilter?

JWT authentication must happen before controllers execute. The filter intercepts every incoming HTTP request, extracts the Authorization header, validates the JWT, extracts the username, loads UserDetails, creates an Authentication object, stores it in SecurityContextHolder, and then allows the request to continue.

Why use a Filter?

A filter sits between the client and the controller. It can inspect, modify, authenticate, or reject requests before they reach application code. This centralizes authentication logic instead of duplicating it in every controller.

Why OncePerRequestFilter?

OncePerRequestFilter guarantees that the filter executes only once for each HTTP request. This prevents duplicate JWT validation during internal forwards, includes, asynchronous dispatches, or error dispatches.

Request Flow

Client -> JwtAuthenticationFilter -> Read Authorization header -> Extract JWT -> Validate JWT -> Extract Username -> Load UserDetails -> Create Authentication -> Store in SecurityContextHolder -> Spring Security Authorization -> Controller -> Service -> Database.

Role of SecurityContextHolder

After successful validation, the Authentication object is stored in SecurityContextHolder. Spring Security then treats the request as authenticated, allowing authorization checks and access to the current user throughout the request.

Role of FilterChain

Calling filterChain.doFilter(request, response) passes control to the next filter and eventually to the controller. Omitting this line stops request processing.

Summary

- JwtAuthenticationFilter authenticates every request carrying a JWT.
- OncePerRequestFilter ensures the authentication logic runs exactly once per request.
- Authentication is established before controllers execute.
- SecurityContextHolder stores the authenticated user for the remainder of the request.